



PROGRAMACIÓN IV

AVANCE PROYECTO – Progreso 1

TEMA: Sistema de monitoreo y análisis de telemetría en videojuego mediante integración no invasiva entre Unity, backend y dashboard web

ESTUDIANTES: Alisson Armas, Samuel Cobo, Julio Mera, Amelia Povea y Jeremy Tomaselly

PROFESOR: PhD (c). Luis Fernando Aguas Bucheli

QUITO – ECUADOR

AVANCE 1

1. Nombre del Equipo, Nombre Creativo del Proyecto y Diseño de Logos

Para abarcar la complejidad técnica de este proyecto integrador, hemos estructurado a nuestro equipo de la siguiente manera. Dark Sky Studios es el nombre de nuestro equipo que por un lado desarrolla el videojuego Cosmic Diner y por otro desarrollará este proyecto que será conocido como EchoConsole. EchoConsole se relaciona directamente con Cosmic Diner, pues se encargará de mostrar en tiempo real información como: cuántos jugadores están conectados y cuántas veces se ha instalado el juego. A continuación, presentamos la identidad de nuestras dos divisiones y sus respectivos productos.

Nombre del equipo: Dark Sky Studios.

Figura 1

Logo Dark Sky Studios



Cuando nos sentamos a definir quiénes somos como desarrolladores, teníamos claro que queríamos huir de los clichés del terror (sangre, monstruos o tipografías rasgadas). Nuestro logo es, literalmente, un pastizal oscuro que ha sido invertido. ¿Por qué? Porque creemos que el verdadero terror no nace de lo grotesco, sino de tomar algo completamente natural y cotidiano, y torcerlo.

A primera vista, el logo parece un simple recuadro oscuro con un trazo abstracto. Sin embargo, al observar detenidamente y darte cuenta de que es el suelo lo que cuelga sobre ti, genera una sutil sensación de vértigo y desorientación. Esa sensación de que "las reglas de la

realidad están mal" es exactamente la firma que queremos dejar en nuestros jugadores: un terror psicológico que se infiltra en tu cabeza antes de que el juego siquiera intente asustarte.

Nombre del Proyecto: EchoConsole

Figura 2

Logo del proyecto EchoConsole



EchoConsole es el producto estrella de nuestra división de datos y el puente directo hacia Cosmic Diner. Es el dashboard web construido en ASP.NET Core que utiliza SignalR para mostrar en tiempo real la información de las sesiones de juego. El nombre nace de un concepto muy específico: cada vez que un jugador inicia una partida o interactúa con nuestro juego Cosmic Diner, genera un "eco" de datos que viaja por la red hasta nuestra consola central.

El logo de EchoConsole consiste en una serie de hexágonos concéntricos con un degradado de neón que se expanden hacia afuera desde un núcleo central brillante. Este diseño abstracto representa visualmente la recepción de señales, como si fuera un radar de alta tecnología capturando los heartbeats (latidos de conexión) que envían las diferentes builds de Unity. Simboliza nuestro servidor central recibiendo, sincronizando y centralizando la información en tiempo real, dándonos control y visibilidad absoluta sobre lo que ocurre en instancias aisladas del juego.

2. Contextualización e Identificación del Problema o Necesidad

Los videojuegos forman parte del día a día. Mientras alguien intenta superar niveles en Candy Crush durante un momento libre, otra persona compite en partidas intensas de Call of Duty buscando la victoria. Esta presencia constante demuestra cómo el gaming se ha integrado en la rutina de millones de personas en todo el mundo. Este fenómeno no solo es evidente en la vida cotidiana, sino también en los datos. “El tamaño del mercado mundial de juegos en línea se valoró en 225,28 mil millones de dólares en 2025 y se proyecta que crecerá de 244,68 mil millones de dólares en 2026” (Fortune Business Insights, 2026). Este crecimiento se debe, en gran medida, a la adopción de nuevas tecnologías y dispositivos. Como el auge de los teléfonos inteligentes y el desarrollo de los juegos en la nube, lo que ha ampliado significativamente el acceso a los videojuegos (Fortune Business Insights, 2026). Como consecuencia, se ha generado un aumento considerable en la cantidad de datos producidos por los usuarios.

En este contexto, surge el análisis de juegos, entendido como “los datos y la información que ayudan a mejorar la toma de decisiones en el diseño de juegos, la monetización, la experiencia de juego y el crecimiento empresarial” (Rosenfelder, 2024). Así, los videojuegos modernos no solo funcionan como productos de entretenimiento, sino también como sistemas distribuidos que generan información valiosa para su mejora continua.

A partir de esta realidad, se vuelve evidente que no basta con recolectar grandes volúmenes de datos, sino que es necesario transformarlos en información útil. Los datos no deben limitarse a una representación visual atractiva, sino que deben aportar valor tanto a los desarrolladores como a los usuarios que interactúan con el sistema. Para una empresa que desarrolla videojuegos, comprender el comportamiento de sus jugadores es fundamental, ya que constituye una forma directa de mantenerse en contacto con ellos y evitar que sus necesidades queden en segundo plano. Sin embargo, no siempre resulta eficiente analizar esta información a través de herramientas técnicas como consolas dentro de entornos de desarrollo, ya que estas suelen ser limitadas y poco accesibles.

En cambio, una plataforma más estructurada permite una visualización más completa y comprensible. En este sentido, como equipo en etapa inicial, la implementación de prácticas de análisis de datos resulta clave para fortalecer el proceso creativo, el mantenimiento del videojuego y su posible crecimiento. Su objetivo principal es ofrecer la mejor experiencia posible al usuario, asegurando que este se sienta escuchado. Como señala Rosenfelder (2024): “la creatividad humana combinada con datos excepcionales produce juegos increíbles”. Además, el mismo autor sugiere que una de las mejores formas de iniciar con el análisis de datos es mediante sistemas que integren tanto información histórica como actual del videojuego; lo que

refuerza la necesidad de contar con herramientas que gestionen estos datos de manera eficiente.

De esta manera, se identifica un problema específico relacionado con la falta de visibilidad sobre aspectos clave del sistema, como el número de jugadores activos y la cantidad de instalaciones del videojuego. Asimismo, existe una dificultad para monitorear múltiples versiones o builds del juego de manera simultánea, lo que limita la capacidad de análisis. Este problema se enmarca en los desafíos propios de los sistemas distribuidos, donde la información se genera desde múltiples fuentes y requiere mecanismos adecuados para su integración y monitoreo.

El análisis de este problema también debe considerar diversos factores contextuales. En el ámbito global, se proyecta que el mercado continúe en expansión, alcanzando “501,91 mil millones de dólares en 2034, exhibiendo una tasa compuesta anual del 9,4%” (Fortune Business Insights, 2026). Este crecimiento implica una distribución masiva de software y una mayor necesidad de monitoreo remoto en diferentes plataformas. Mientras que, desde el punto de vista social, se reconoce que “para maximizar los efectos positivos de los videojuegos, es importante centrarse en cómo los diferentes tipos de jugadores desarrollan sus hábitos de juego” (Kaufmann, 2022). Esto resalta la importancia de comprender el comportamiento de los usuarios y adaptar las experiencias digitales en función de sus necesidades.

En el ámbito económico, la toma de decisiones basada en datos se ha vuelto esencial. Como indica Pereira (2023), “en esta era digital, la información precisa y confiable es esencial para guiar las decisiones estratégicas [...] importancia de estos elementos es crucial para que las industrias prosperen en el entorno altamente competitivo del e-commerce”. Esto demuestra que el análisis de métricas no solo permite evaluar el éxito de un producto, sino también optimizar los recursos durante su desarrollo. Por otro lado, en el factor ambiental, aunque de forma más indirecta: “Microsoft estima que una consola de videojuegos moderna promedio emite alrededor de 72 kilogramos de dióxido de carbono al año” (Safdie, 2025). Esto evidencia la necesidad de gestionar eficientemente los recursos digitales y evitar la sobrecarga de sistemas mediante un monitoreo adecuado.

En cuanto al factor cultural, los videojuegos se han consolidado como un fenómeno diverso. Para algunas personas, representan una forma de relajación tras una jornada extensa; para otras, constituyen un medio de socialización o incluso la demostración de habilidades (Kaufmann, 2022). Esta diversidad en los patrones de uso refuerza la importancia de analizar el comportamiento de los usuarios.

Frente a este panorama, la implementación de una página web como herramienta de monitoreo se presenta como una solución adecuada. Esta permite centralizar la información proveniente del videojuego. Además, su integración como una capa independiente dentro del proyecto garantiza que no interfiera con el gameplay ni con los sistemas existentes. De acuerdo con Rosenfelder (2024), una plataforma como esta:

Ayuda a mejorar todos los aspectos del juego, desde la publicidad hasta la jugabilidad. Cuando se usa correctamente, el análisis de datos ayuda a que los juegos sean más divertidos, lo que reduce la pérdida de clientes y hace que el juego sea rentable.

Esto respalda la utilidad de este tipo de soluciones en el contexto del desarrollo de videojuegos.

En síntesis, la propuesta responde a un problema real relacionado con la falta de herramientas accesibles para el monitoreo y análisis de datos en videojuegos. Cuenta con sustento teórico que respalda la importancia del análisis de datos en la industria digital y considera diversos factores contextuales que influyen en su relevancia. Además, aporta valor al permitir una mejor comprensión del comportamiento de los usuarios sin afectar el funcionamiento del sistema, lo que la convierte en una solución pertinente y viable.

3. Módulos y Sub-funcionalidades de la Solución Computacional

La solución computacional se plantea como un sistema de monitoreo y administración del videojuego, el proyecto estará dividido en dos frentes tecnológicos:

- Una aplicación web desarrollada en ASP.NET Core MVC, encargada del panel administrativo, gestión de registros y visualización de información.
- Una aplicación en .NET MAUI, destinada a la supervisión rápida, consulta de estados y operaciones esenciales desde dispositivos móviles o de escritorio.

Unity únicamente enviará eventos mínimos al backend, mientras que el manejo administrativo, la persistencia de datos y los procesos CRUD estarán centralizados en la parte ASP.NET MVC, la app MAUI y la base de datos SQL Server.

Módulo 1: Gestión de Usuarios y Roles

Objetivo del módulo: Administrar las cuentas del sistema para administradores, supervisores y operadores que utilizarán la web y la aplicación MAUI.

Tabla 1

Tabla de funcionalidades CRUD del módulo de gestión de usuarios y roles

Operación CRUD	Sub-funcionalidades
Crear	Crear usuario del sistema Registrar rol del usuario Crear credenciales de acceso Registrar estado inicial del usuario
Leer	Consultar listado de usuarios Buscar usuario por nombre, correo o rol Ver detalle de usuario Consultar historial básico de acceso
Actualizar	Editar datos personales del usuario Cambiar rol o permisos Cambiar contraseña Actualizar estado del usuario
Eliminar	Eliminar usuario Eliminar rol asignado erróneamente Eliminar cuentas de prueba Eliminar usuarios inactivos u obsoletos

Funcionalidades en ASP.NET Core MVC

- Mantenimiento completo de usuarios.
- Gestión de roles y permisos.
- Vista de tabla con filtros y búsqueda.
- Control de acceso por perfil

Funcionalidades en .NET MAUI

- Inicio de sesión.
- Consulta rápida de usuarios.
- Edición de perfil propio.
- Cambio de contraseña.
- Activación o desactivación rápida de usuarios.

Módulo 2. Gestión de Juegos, Builds y Versiones

Objetivo del módulo: Administrar los videojuegos monitoreados, sus builds, versiones y metadatos técnicos para identificar desde qué versión se conectan los clientes Unity.

Tabla 2

Tabla de funcionalidades CRUD del módulo de juegos, builds y versiones

Operación CRUD	Sub-funcionalidades
Crear	Registrar videojuego Registrar build nueva Registrar versión del sistema Registrar plataforma o entorno de ejecución
Leer	Consultar lista de juegos registrados Consultar lista de builds Ver detalle de versión Filtrar builds por estado, fecha o versión
Actualizar	Editar nombre o descripción del juego Actualizar número de versión Cambiar estado de la build Editar notas técnicas o de despliegue
Eliminar	Eliminar videojuegos de prueba Eliminar builds obsoletas Eliminar versiones duplicadas Eliminar registros mal cargados

Funcionalidades en ASP.NET Core MVC

- Panel completo de administración de juegos y builds.
- Tabla de versiones y estados.
- Formularios CRUD completos.
- Asociación de builds con instalaciones y sesiones

Funcionalidades en .NET MAUI

- Consulta rápida de builds activas.
- Registro rápido de una build nueva.
- Actualización de estado de build.
- Visualización de versión conectada por cliente

Módulo 3. Gestión de Dispositivos e Instalaciones

Objetivo del módulo: Controlar las instalaciones del videojuego en equipos cliente para conocer cuántas instalaciones registradas existen y con qué dispositivos están asociadas.

Tabla 3

Tabla de funcionalidades CRUD del módulo de gestión de dispositivos e instalaciones

Operación CRUD	Sub-funcionalidades
Crear	Registrar instalación del juego Registrar dispositivo o equipo Asociar instalación con una build Registrar fecha de primera ejecución
Leer	Consultar instalaciones registradas Ver detalle de un dispositivo Buscar instalaciones por equipo, build o estado Consultar total de instalaciones activas e históricas
Actualizar	Editar alias del dispositivo Actualizar estado de instalación Reasignar instalación a otra build Actualizar última fecha de actividad
Eliminar	Eliminar instalación duplicada Eliminar dispositivo mal registrado Eliminar registros de prueba Eliminar instalaciones obsoletas

Funcionalidades en ASP.NET Core MVC

- Gestión completa de instalaciones.
- Vista por equipo y por versión.
- Filtros por estado y fecha.
- Panel de detalle de cada instalación

Funcionalidades en .NET MAUI

- Consulta rápida de instalaciones.
- Registro o vinculación rápida de dispositivo.
- Edición de alias o estado.
- Eliminación o depuración de registros de prueba.

Módulo 4. Gestión de Sesiones y Monitoreo en Tiempo Real

Objetivo del módulo: Registrar y monitorear las sesiones activas de los jugadores o clientes conectados, mostrando cuántos están en línea en tiempo real.

Tabla 4

Tabla de funcionalidades CRUD del módulo de gestión de sesiones y monitoreo en tiempo real

Operación CRUD	Sub-funcionalidades
Crear	Crear sesión al iniciar el juego Registrar hora de conexión Registrar heartbeat o señal de presencia Registrar evento de apertura de cliente
Leer	Consultar sesiones activas Ver historial de sesiones Consultar usuarios conectados en tiempo real Filtrar sesiones por fecha, build o dispositivo
Actualizar	Actualizar heartbeat de sesión Cambiar estado de sesión Actualizar tiempo de última actividad Actualizar observaciones de monitoreo
Eliminar	Eliminar sesiones de prueba Eliminar sesiones corruptas Eliminar registros temporales duplicados Eliminar sesiones cerradas por depuración administrativa

Funcionalidades en ASP.NET Core MVC

- Dashboard en tiempo real.
- Tabla de sesiones activas.
- Vista histórica de conexiones.
- Indicador de jugadores conectados simultáneamente.
- Cierre manual o invalidación de sesión

Funcionalidades en .NET MAUI

- Visualización de conectados en tiempo real.

- Alertas de conexión o desconexión.
- Consulta rápida de sesión por dispositivo.
- Cierre o invalidación administrativa de sesión.

Módulo 5. Gestión de Alertas, Incidencias y Reportes

Objetivo del módulo: Gestionar alertas, incidencias y reportes administrativos del sistema, permitiendo identificar errores, desconexiones y eventos relevantes.

Tabla 5

Tabla de funcionalidades CRUD del módulo de alertas, incidencias y reportes

Operación CRUD	Sub-funcionalidades
Crear	Crear alerta del sistema Registrar incidencia Crear reporte manual Configurar regla de alerta
Leer	Consultar alertas activas Ver historial de incidencias Consultar reportes generados Filtrar alertas por prioridad, fecha o estado
Actualizar	Editar descripción de alerta Cambiar prioridad Actualizar estado de incidencia Actualizar observaciones del reporte
Eliminar	Eliminar alertas de prueba Eliminar incidencias duplicadas Eliminar reportes temporales Eliminar reglas de alerta obsoletas

Funcionalidades en ASP.NET Core MVC

- Panel administrativo de alertas.
- Gestión completa de incidencias.
- Generación y consulta de reportes.
- Filtros avanzados por prioridad y estado.

Funcionalidades en .NET MAUI

- Notificaciones rápidas de alertas.
- Registro inmediato de incidencias.
- Consulta de reportes resumidos.
- Cambio rápido de estado de incidencia.

4. Diagramas C4 – Nivel 1 y Nivel 2

La elaboración de los diagramas C4 representa un paso clave para describir de forma clara y estructurada la arquitectura del sistema. A través de esta metodología, se busca ofrecer una visión progresiva del proyecto, permitiendo comprender tanto su contexto general como la organización interna de sus componentes principales. Este enfoque facilita la comunicación entre los integrantes del equipo y asegura una mejor comprensión del funcionamiento del sistema.

En esta sección, se desarrollan los diagramas correspondientes a los niveles 1 y 2 del modelo C4. El diagrama de contexto (Nivel 1) presenta el sistema en su totalidad, identificando a los usuarios y los sistemas externos con los que interactúa. Por su parte, el diagrama de contenedores (Nivel 2) profundiza en la estructura interna, mostrando las aplicaciones, bases de datos, APIs y servicios que conforman la solución, así como las relaciones y flujos de comunicación entre ellos.

Figura 3

Diagrama de nivel 1

Enlace: [PROIV Proyecto_C4_Nivel1.drawio](#)

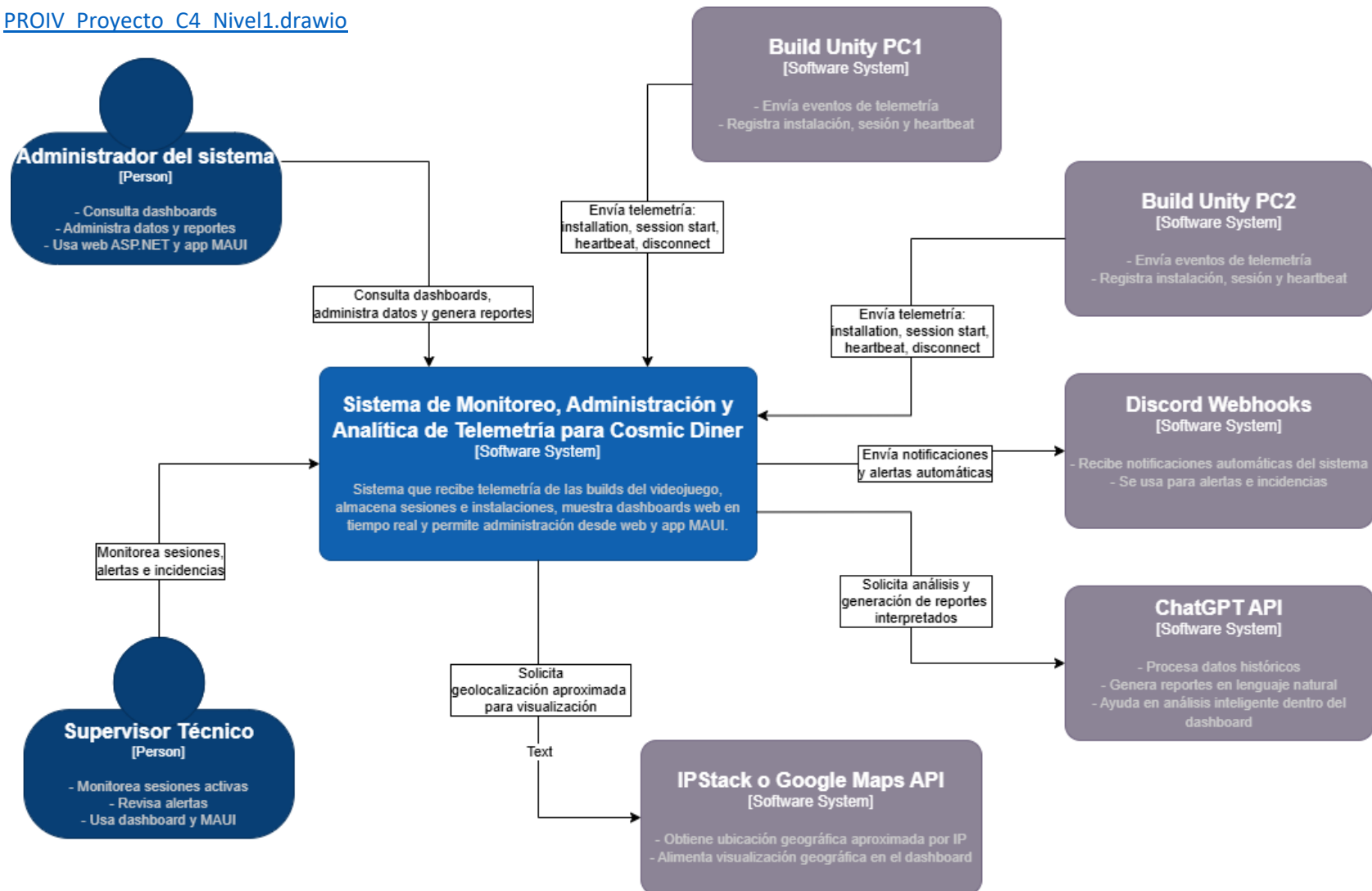
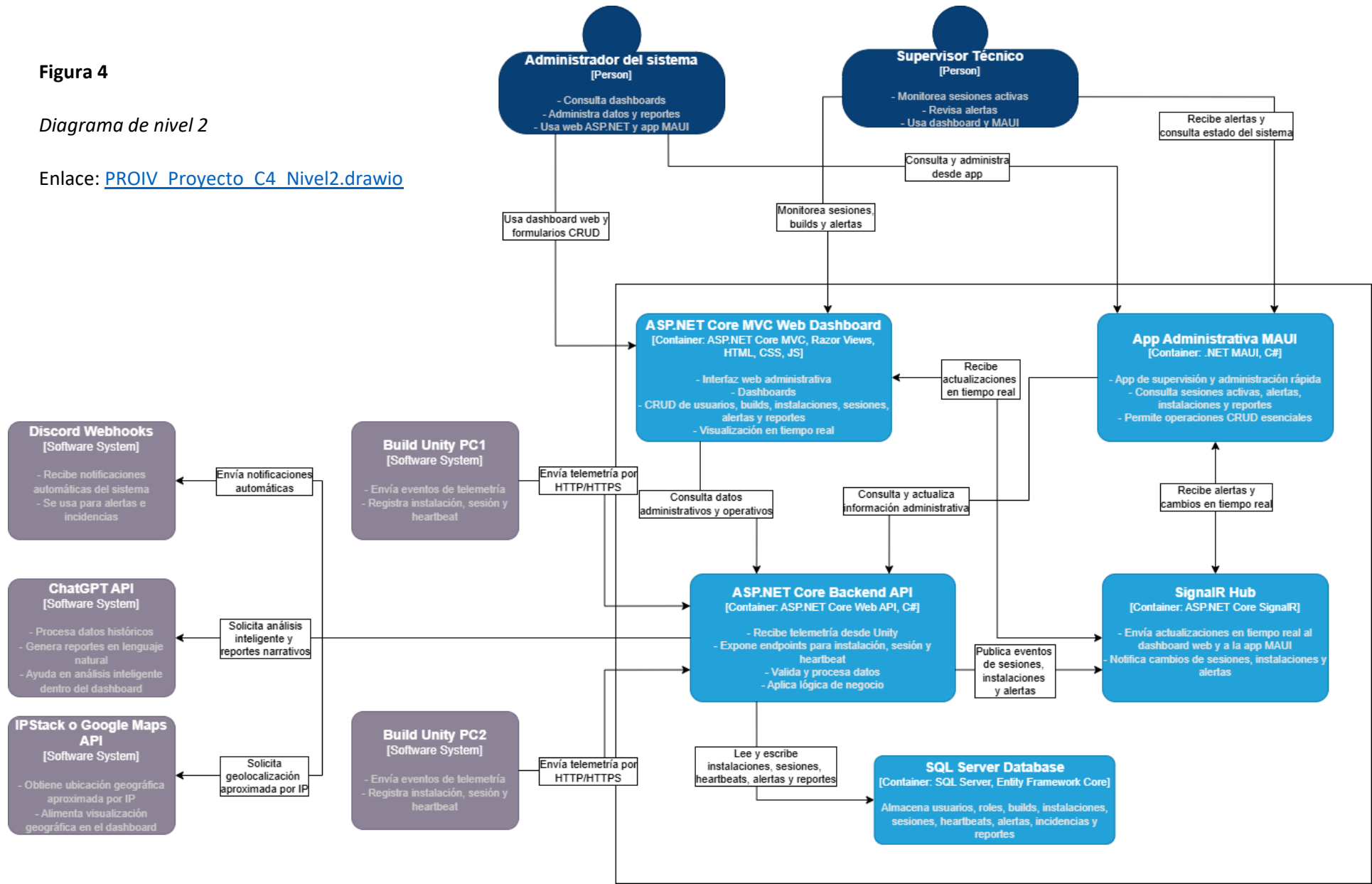


Figura 4

Diagrama de nivel 2

Enlace: [PROIV Proyecto C4 Nivel2.drawio](#)



Estos diagramas no se generan automáticamente a partir del código, sino que se construyen de manera intencional para reflejar la arquitectura global del sistema y la distribución de responsabilidades entre sus componentes. De esta forma, permiten visualizar de manera clara cómo se organiza la solución y cómo interactúan sus diferentes partes.

5. Funcionalidades Plus y Valor Agregado

En el desarrollo del proyecto, no basta con implementar únicamente funcionalidades básicas de gestión de datos, como las operaciones CRUD. Por ello, se plantea la incorporación de funcionalidades plus que aporten valor agregado a la solución, fortaleciendo tanto la experiencia del usuario como la capacidad de análisis del sistema. Estas funcionalidades permiten que la plataforma no solo almacene y muestre información, sino que también la interprete, la presente de forma interactiva y genere aportes útiles para la toma de decisiones. En este sentido, se consideran elementos como:

Implementación de Dashboards con Análisis de Retención y Mapas de Calor

Transforma los datos planos en inteligencia operativa mediante el uso de gráficos interactivos que segmentan el uso por versión del build y calculan el tiempo promedio de sesión por cada instalación registrada. Esta funcionalidad es vital para el proyecto porque nos permite como desarrolladores identificar con precisión qué versiones del juego son más estables o atractivas, facilitando la toma de decisiones basada en datos reales sobre cuándo lanzar parches o actualizaciones de contenido sin comprometer el rendimiento.

Sistema de Alertas

Un Sistema de Alertas por "Caída de Sesiones" aprovecha la lógica de heartbeats ya estructurada en la arquitectura para disparar alertas automáticas en el dashboard web de la PC3 si las desconexiones inesperadas superan un umbral técnico definido. Esta capacidad mejora drásticamente la eficiencia del sistema, ya que permite detectar en tiempo real problemas críticos como fallos de red en la LAN o errores de software en las builds de Unity, permitiendo una respuesta técnica inmediata antes de que afecte a más usuarios.

Recompensas

La funcionalidad de Recompensas por "Lealtad de Instalación" es una Gamificación Ligera, ya que utiliza los campos de la entidad Installation, específicamente el InstallationId y la fecha FirstSeenUtc, para que el backend de ASP.NET Core gestione incentivos o desbloqueables digitales al cumplirse hitos de tiempo de juego acumulado. Este valor agregado mejora significativamente la experiencia del usuario al reconocer su permanencia y fidelidad, incentivando el uso continuo del videojuego de manera orgánica y sin tener que alterar la mecánica base del gameplay o los sistemas existentes.

De esta manera, estas funcionalidades adicionales no solo enriquecen el sistema desde un punto de vista técnico, sino que también mejoran la eficiencia operativa y permiten una interacción más intuitiva y efectiva con la información. En conjunto, contribuyen a que la solución propuesta sea más completa, escalable y alineada con las demandas actuales de los sistemas digitales modernos.

APIs aplicables

La integración de IPStack o Google Maps API nos permite que, al recibir el evento SessionStart en el backend, se capture la IP del cliente para determinar su ubicación geográfica aproximada en una fase de expansión más allá de la red LAN. El uso de estas APIs permite desplegar en el dashboard un mapa interactivo que visualice la distribución global de los jugadores activos en tiempo real. El impacto para el proyecto es profundo, ya que proporciona una herramienta visual poderosa para estrategias de marketing y permite una mejor planificación de la logística de servidores según la concentración geográfica de los usuarios.

La ChatGPT API puede integrarse en el dashboard web para procesar el histórico de sesiones, instalaciones y heartbeats almacenados en SQL Server. Mediante el procesamiento de lenguaje natural, los administradores pueden realizar consultas complejas sobre el comportamiento de los jugadores y recibir informes interpretados automáticamente sobre tendencias de uso o errores frecuentes. Esta integración potencia la toma de decisiones estratégica, permitiendo que incluso personal no técnico comprenda profundamente el estado y rendimiento del ecosistema de las builds de Unity.

La API de Discord Webhooks funciona como el canal de comunicación proactivo del servidor central en PC3, permitiendo que el sistema envíe notificaciones instantáneas a un canal de chat mediante peticiones HTTP simples cada vez que el backend detecta una anomalía o un evento relevante en la base de datos. Esta herramienta facilita la implementación del sistema

de alertas inteligentes, ya que puede informar automáticamente si una sesión en PC1 o PC2 ha perdido sus heartbeats o si se ha registrado una nueva instalación, permitiendo una reacción técnica inmediata sin necesidad de monitorear el dashboard constantemente. Al ser una solución gratuita y de fácil integración mediante UnityWebRequest en el cliente o peticiones estándar en .NET, aporta un alto valor agregado al proyecto al mejorar la eficiencia en la detección de fallos de red o errores de software, manteniendo la premisa de no interferir con el rendimiento del gameplay ni alterar la arquitectura funcional.

6. Selección de la Mejor Alternativa

Para lograr la integración exitosa entre el entorno interactivo de Cosmic Diner y el ecosistema de monitoreo web, nos enfrentamos al desafío arquitectónico de extraer información en tiempo real desde un motor gráfico complejo sin perjudicar el rendimiento del juego ni exponer la infraestructura a vulnerabilidades. Para resolver esta problemática, evaluamos dos enfoques de diseño diametralmente opuestos antes de tomar una decisión fundamentada.

Alternativa A: Arquitectura Acoplada con Conexión Directa a la Base de Datos

La primera alternativa que consideramos, representa un enfoque rudimentario pero frecuentemente utilizado en etapas iniciales de desarrollo. Consiste en establecer una conexión directa y bidireccional entre los clientes del videojuego desarrollados en Unity y el motor de base de datos SQL Server. Bajo este esquema estructural, los propios scripts encargados de la lógica interna del juego tendrían la responsabilidad adicional de importar librerías nativas de acceso a datos para abrir conexiones y ejecutar sentencias SQL directas cada vez que se requiera registrar un inicio de sesión o enviar un latido de estado.

La consecuencia operativa de este diseño es que obliga al cliente gráfico a gestionar procesos de red pesados que habitualmente son de naturaleza sincrónica. Esto, inevitablemente provocaría micro interrupciones en el ciclo de renderizado del motor gráfico y caídas perceptibles en los fotogramas por segundo (FPS), arruinando la atmósfera de inmersión y tensión que requiere un juego de terror psicológico.

Sumado a las deficiencias en el cliente, para que la página web pudiera visualizar a los jugadores conectados en vivo, el dashboard se vería forzado a implementar una técnica de actualización basada en Polling. Esto significa que la aplicación web tendría que consultar a la

base de datos de manera automatizada y repetitiva cada escasos segundos para verificar si existe nueva información, saturando los recursos del servidor con tráfico innecesario incluso en momentos donde no hay ningún usuario jugando.

Alternativa B: Arquitectura Cliente-Servidor Desacoplada mediante API REST y SignalR

Como segunda opción, planteamos una arquitectura moderna y completamente desacoplada que delega las responsabilidades pesadas a un servidor intermediario, siendo esta la solución técnica que hemos adoptado para el proyecto. En este escenario, el videojuego conserva intacta su arquitectura original y se mantiene puramente enfocado en el procesamiento de físicas, inteligencia artificial y gráficos. Todo esto, limitándose a empaquetar eventos operativos mínimos en pequeños archivos de texto formato JSON que son enviados de forma ligera y asíncrona mediante peticiones HTTP estándar.

Todas estas peticiones son recibidas por un servidor centralizado (PC3) que aloja una API desarrollada en el robusto framework ASP.NET Core, el cual asume el rol de único mediador autorizado para interactuar con la base de datos SQL Server utilizando Entity Framework Core. Así se garantiza que la estructura y las credenciales de los datos permanezcan totalmente invisibles e inaccesibles para las instancias del juego. Simultáneamente, para resolver el problema de la visualización en la página web sin incurrir en el desperdicio de recursos del Polling, la API central hace uso de la tecnología SignalR para establecer un canal de comunicación persistente que le permite "empujar" las actualizaciones directamente hacia el dashboard web EchoConsole en el instante exacto en que se registra un nuevo evento, logrando un monitoreo verdaderamente en tiempo real, elegante y sin sobrecargar la red local.

Análisis comparativo utilizando criterios de calidad de la norma ISO/IEC 25010.

Para fundamentar nuestra decisión técnica con rigor académico y profesional, sometimos ambas alternativas arquitectónicas a un proceso de evaluación guiado por los estándares de calidad de la norma ISO/IEC 25010. Considerando la naturaleza híbrida de nuestro proyecto, hemos seleccionado tres características críticas de este modelo que impactan directamente en la viabilidad de un ecosistema que combina motores gráficos exigentes con herramientas web de sincronización en tiempo real: Eficiencia de Desempeño, Seguridad y Mantenibilidad.

Eficiencia de Desempeño

Al analizar este primer criterio, la Alternativa A presenta deficiencias operativas severas que resultan incompatibles con los estándares de calidad del desarrollo de videojuegos. Al obligar al cliente de Unity a gestionar conexiones directas con SQL Server, se introducen operaciones de red que a menudo se ejecutan de manera sincrónica, compitiendo directamente por los recursos del procesador y bloqueando el hilo principal de ejecución. Esto se traduce inevitablemente en molestas caídas de fotogramas por segundo (FPS) que rompen por completo la atmósfera de inmersión del jugador. Adicionalmente, la técnica de Polling requerida por el dashboard en esta opción saturaría nuestro servidor central con un volumen constante de consultas vacías, generando tráfico de red innecesario incluso cuando no hay jugadores activos.

En marcado contraste, la Alternativa B sobresale al utilizar peticiones HTTP asíncronas y sumamente ligeras que Unity puede despachar en segundo plano sin alterar en lo absoluto el flujo del juego. Esta alta eficiencia se maximiza en el componente web gracias a la implementación de SignalR, el cual instaura un modelo orientado a eventos donde el servidor únicamente consume recursos para transmitir información hacia el dashboard cuando ocurre un cambio real en las sesiones, optimizando drásticamente el uso del ancho de banda y la capacidad de procesamiento de nuestra máquina central.

Seguridad

En lo que respecta a la protección de la información, la Alternativa A representa un nivel de riesgo crítico e inaceptable para cualquier entorno de producción. Para que las instancias locales del videojuego puedan comunicarse directamente con la base de datos, es un requisito técnico incrustar la cadena de conexión —que contiene la dirección del servidor, el usuario y la contraseña de acceso— directamente dentro del código fuente del cliente. Esto abre una brecha de vulnerabilidad extrema donde cualquier usuario con conocimientos básicos podría descompilar la aplicación de Unity, extraer nuestras credenciales y obtener control total para alterar, secuestrar o destruir la integridad de nuestra información.

Frente a este panorama riesgoso, la Alternativa B blindada completamente nuestra infraestructura al establecer la API de ASP.NET Core como una barrera de abstracción infranqueable. En este diseño, la base de datos vive resguardada dentro del servidor en PC3 sin exposición directa al exterior, limitando la interacción de los clientes exclusivamente a endpoints públicos de la API que están estrictamente programados para recibir estructuras de

datos muy específicas, lo cual anula por completo el riesgo de inyecciones SQL o accesos no autorizados a la arquitectura profunda del sistema.

Mantenibilidad y Modificabilidad

Finalmente, al evaluar la capacidad del sistema para evolucionar en el tiempo, la Alternativa A fomenta un acoplamiento técnico sumamente deficiente al mezclar la lógica de persistencia de datos con las mecánicas fundamentales del gameplay, como las físicas o la inteligencia artificial. Este enredo lógico produce un código frágil donde cualquier mínimo cambio en la estructura de una tabla en la base de datos obligaría al equipo a reescribir, depurar y generar una nueva compilación completa del videojuego.

Por el contrario, la Alternativa B brilla gracias a su modularidad fundamentada en el principio de separación de responsabilidades. Al encapsular toda la lógica de telemetría y comunicación de red en un directorio completamente aislado dentro de Unity, garantizamos que el desarrollo del juego base progrese sin interferencias tecnológicas. Esta arquitectura limpia y desacoplada nos otorga la libertad absoluta de actualizar la versión de nuestra base de datos, reestructurar la lógica de la API central o incluso migrar todo el servidor backend hacia plataformas en la nube en el futuro, teniendo la certeza técnica de que el cliente del videojuego no sufrirá ninguna ruptura funcional ni requerirá modificaciones en su código fuente.

Justificación de la selección

Con base en el exhaustivo análisis comparativo detallado en los apartados anteriores, nuestro equipo de desarrollo ha determinado unánimemente que la Alternativa B se erige como la única opción viable y profesional para sustentar las exigencias técnicas de este proyecto integrador. Esta decisión definitiva se fundamenta en tres pilares esenciales de la ingeniería de software moderna que garantizan el éxito tanto del videojuego como de la plataforma de monitoreo:

Eficiencia: Preservación de la Eficiencia y la Inmersión del Usuario

La primera gran justificación radica en la protección absoluta de la experiencia del usuario final, dado que en un videojuego de terror psicológico y supervivencia como Cosmic Diner, la inmersión lo es todo y no podemos permitirnos que un proceso en segundo plano sacrifique el rendimiento gráfico o provoque interrupciones abruptas. Al delegar todo el trabajo pesado, la validación de reglas de negocio y el guardado persistente de datos a nuestra API

central en ASP.NET Core, garantizamos que el cliente de Unity opere con exactamente la misma fluidez que si no tuviera ningún sistema de telemetría integrado; simultáneamente, la inclusión de SignalR nos asegura que la división de PulseTrack Technologies pueda observar los latidos de conexión en el dashboard web de manera instantánea y reactiva, sin desperdiciar recursos de red en consultas repetitivas.

Escalabilidad: Alta Escalabilidad y Preparación para Entornos Cloud

En segundo lugar, respaldamos esta decisión por la altísima capacidad de crecimiento que nos proporciona una arquitectura fundamentada en el desacoplamiento de servicios. Aunque nuestra implementación inicial para esta fase académica operará bajo un entorno de pruebas controlado en una red local (LAN) utilizando tres computadoras, el hecho de contar con un servidor intermediario independiente nos otorga una enorme ventaja estratégica para el futuro; si el proyecto demanda una expansión, podremos migrar toda nuestra infraestructura web y la base de datos hacia proveedores en la nube como Microsoft Azure o Amazon Web Services, logrando un alcance global sin necesidad de reestructurar la lógica del videojuego, ya que los clientes de Unity únicamente requerirán una simple actualización en la dirección URL hacia la que envían sus peticiones HTTP.

Simplicidad del sistema: Arquitectura Limpia y Optimización del Flujo de Trabajo

Finalmente, desde una perspectiva puramente organizativa y de gestión de proyectos, seleccionamos esta alternativa porque promueve un entorno de desarrollo colaborativo mucho más ordenado, modular y libre de conflictos. Al aplicar un principio estricto de separación de responsabilidades, logramos que nuestra división encargada del desarrollo interactivo pueda concentrarse exclusivamente en perfeccionar las mecánicas de juego, el diseño de niveles y el comportamiento de la entidad enemiga sin preocuparse por la persistencia de datos; de manera paralela, el equipo de backend adquiere la libertad total para iterar, parchear y fortalecer el dashboard EchoConsole o modificar las tablas en SQL Server sin temor a "romper" el código del videojuego, consolidando así un producto de software maduro, profesional y verdaderamente alineado con los estándares actuales de la industria tecnológica.

7. Aplicación de IA

La Inteligencia Artificial puede mejorar significativamente la solución del proyecto al actuar como una capa de automatización de procesos y análisis de datos transformando la

información técnica en conocimiento estratégico. En lugar de limitarse a almacenar datos de sesiones en SQL Server, la IA puede procesar el historial de telemetría para identificar patrones de comportamiento y predecir posibles fallos en las builds de Unity antes de que ocurran, optimizando así la estabilidad del sistema. Además, mediante la integración de modelos de lenguaje, el proyecto puede ofrecer una interfaz de consulta en lenguaje natural para el administrador, facilitando la toma de decisiones basada en reportes narrativos y diagnósticos precisos sobre el uso del software sin requerir una revisión manual de las bases de datos.

Asimismo, la IA nos permite implementar predicciones y recomendaciones personalizadas que elevan la experiencia del usuario de manera no invasiva. Al analizar los datos de Installation y GameSession, un motor de IA puede determinar el perfil de uso de cada jugador y sugerir incentivos o recompensas digitales de forma dinámica y automática, ajustándose al nivel de compromiso de cada instalación. Este enfoque no solo mejora la eficiencia del sistema al autogestionar la lógica de fidelización, sino que también garantiza que la arquitectura se mantenga ligera y escalable.

8. Diseño de Mockups

El diseño de mockups constituye una etapa fundamental en el desarrollo del proyecto, ya que permite representar de manera visual la estructura y el funcionamiento del sistema antes de su implementación. A través de Figma, se busca construir prototipos de las pantallas principales que reflejen cómo interactuarán los usuarios con la plataforma, facilitando la comprensión de la interfaz y la experiencia de uso.

De esta manera, los mockups permiten validar ideas, identificar posibles mejoras en la usabilidad y establecer una guía clara para el desarrollo del sistema, contribuyendo a una solución más intuitiva, estructurada y centrada en el usuario.

Enlace a figma:

<https://www.figma.com/proto/21jrQQLndvV8pByvjEKsoK/Untitled?node-id=1-6&p=f&t=sMv7Pf4cxnCGHN6S-1&scaling=min-zoom&content-scaling=fixed&page-id=0%3A1>

9. Declaratoria de Uso de IA

En el presente proyecto, la Inteligencia Artificial se aplica específicamente en la página web administrativa desarrollada en ASP.NET Core MVC, alojada en la PC3, la cual funciona como servidor central del sistema. Su uso no está orientado al gameplay del videojuego ni a la lógica

interna de Unity, sino al procesamiento inteligente de los datos de telemetría recibidos desde las builds ejecutadas en PC1 y PC2.

Además, es importante señalar que, en esta etapa del proyecto, la IA también ha sido utilizada como una herramienta de apoyo en la fase de diseño y planificación de la solución. En particular, ha contribuido al planteamiento de la arquitectura del sistema, incluyendo la definición de su estructura, la organización de funcionalidades y la identificación de módulos adecuados. Este uso ha permitido orientar el desarrollo hacia un enfoque más estructurado y profesional, considerando que tanto la intención del proyecto como el videojuego desarrollado son reales. Este aporte corresponde al presente avance del proyecto.

La IA se integra en el dashboard web como una capa de análisis y asistencia sobre la información almacenada en SQL Server, permitiendo transformar datos técnicos en reportes comprensibles y útiles para la toma de decisiones. En particular, la IA se aplica en las siguientes partes de la solución web:

Partes de la página web donde se aplica la IA

Apoyo en el diseño arquitectónico del sistema (fase de avance actual)

Durante este avance, la IA ha sido utilizada como herramienta de apoyo para definir la arquitectura de la solución. Esto incluye la estructuración del sistema, la separación en capas, la identificación de módulos funcionales y la organización de las interacciones entre el videojuego, el backend y la página web. Este uso no forma parte del sistema en producción, sino del proceso de diseño, y ha permitido establecer una base sólida para el desarrollo del proyecto

Análisis inteligente del historial de sesiones

La IA procesa los registros de GameSession, Installation y Heartbeat almacenados en la base de datos para identificar patrones de uso, tiempos promedio de sesión, frecuencia de conexión y estabilidad de cada build del videojuego. De esta manera, el dashboard no solo muestra datos planos, sino que también entrega una interpretación automatizada del comportamiento del sistema.

Generación de reportes automáticos en lenguaje natural

Dentro del panel web, la IA permite convertir los datos técnicos en reportes narrativos comprensibles para el administrador. Por ejemplo, en lugar de revisar manualmente tablas de

sesiones o instalaciones, el sistema puede generar resúmenes como: cuál build presenta mayor estabilidad, qué instalación registra más tiempo de uso o en qué momento se detectaron más desconexiones inesperadas.

Detección de anomalías y apoyo en alertas inteligentes

La IA se aplica como apoyo en la detección de comportamientos anómalos dentro del sistema web, especialmente en casos como caídas repetidas de sesiones, pérdida de heartbeats o incrementos inusuales en desconexiones. Esto fortalece el sistema de alertas del dashboard, permitiendo una respuesta más rápida y mejor fundamentada por parte del administrador.

Consultas administrativas en lenguaje natural

La página web incorpora el uso de IA para que el administrador pueda realizar consultas complejas mediante lenguaje natural. En lugar de depender únicamente de filtros manuales o consultas SQL, el usuario puede solicitar interpretaciones sobre el comportamiento de los jugadores, la estabilidad de las builds o el nivel de actividad registrado, recibiendo respuestas automatizadas y contextualizadas.

Predicción y apoyo a decisiones técnicas

La IA puede utilizar el historial acumulado de conexiones, instalaciones y sesiones para inferir tendencias de uso y advertir posibles fallos futuros en determinadas versiones del videojuego. Esta funcionalidad aporta valor estratégico al proyecto, ya que ayuda a decidir cuándo mantener una build, corregirla o lanzar una actualización con base en evidencia operativa.

Herramienta de IA utilizada

La herramienta de IA utilizada en el proyecto es la API de ChatGPT, la cual ya ha sido empleada tanto en la fase de diseño como en la definición de la arquitectura de la solución, y se propone además su integración en el backend de ASP.NET Core MVC. Esta API se conecta al sistema web para procesar los datos consolidados en SQL Server y generar respuestas, interpretaciones y reportes útiles para el administrador del dashboard.

Porcentaje aproximado de contribución de IA en la solución

Se estima que la Inteligencia Artificial aporta aproximadamente un 20% a 25% de la funcionalidad total del sistema, concentrándose exclusivamente en la capa web de análisis, interpretación y apoyo a la toma de decisiones. El resto de la solución corresponde a la arquitectura desarrollada en Unity, ASP.NET Core MVC, SignalR, SQL Server y .NET MAUI, así como a la lógica de telemetría, persistencia, monitoreo y administración implementada por el equipo.

Transparencia y ética en el uso de IA

El uso de IA en este proyecto se realiza de forma transparente, responsable y ética, ya que su función se limita a complementar el análisis de datos dentro del dashboard web y no a reemplazar la lógica principal del sistema ni el trabajo técnico de los desarrolladores. La IA no toma decisiones autónomas sobre el funcionamiento del videojuego, ni altera directamente las mecánicas de Unity, sino que actúa como una herramienta de apoyo para interpretar información, detectar tendencias y facilitar la comprensión del estado general del sistema.

Bibliografía

Fortune Business Insights. (2026, 6 abril). *Tamaño, participación y crecimiento del mercado de juegos en línea | Pronóstico [2034]*. <https://www.fortunebusinessinsights.com/es/online-gaming-market-112179>

Kaufmann, D. (2022, 17 junio). Perfiles de jugadores: Entendiendo la psicología de los videojuegos. Dr. Gameology. <https://drgameology.com/gamer-personas-understanding-game-psychology/>

Pereira, P. (2023, 13 julio). La Importancia de los Datos Precisos en el Comercio Electrónico: Impulsando Decisiones Estratégicas - Lett Digital. Lett Digital. <https://lett.digital/es/product-updates/la-importancia-de-los-datos-precisos-en-el-e-commerce-impulsando-decisiones-estrategicas-experiencia-del-cliente-y-reputacion-de-la-marca/>

Rosenfelder, S. (2024, 1 enero). Análisis de datos en videojuegos: por qué importan las cifras. AppsFlyer. <https://www.appsflyer.com/blog/measurement-analytics/gaming-analytics/>

Safdie, S. (2025, 31 octubre). The Environmental Impact of Video Games. Greenly. <https://greenly.earth/en-us/leaf-media/data-stories/the-environmental-impact-of-video-games>